Frontend Take-Home: Annotation Activity Console

Time budget: about two days. Don't gold-plate this. We'd rather see correct, well-reasoned core work than a half-finished pile of features.

What we're looking for: how you take backend responses and a live stream and turn them into typed state and correct, interactive UI. Visual design is not graded. Use plain Tailwind, the default look is completely fine. We're grading the data path, the state, the types, the correctness, and your reasoning.

You can use AI tools. We assume you will. But in the interview you'll walk us through this code, justify every non-obvious decision, and make live changes, including inside your own state and data layer. So build something you actually understand.

The scenario

A generic activity console. Annotators work through tasks (image, audio, text), and you're building an internal view of those tasks: their status, who's working on them, updating in real time as events come in. Each task also has an AI-generated summary that the backend streams to you on demand. This is a representative exercise, not our actual product, but it mirrors the work: messy backend data, a live event stream, and a streamed model response rendered as markdown.

We've given you a mock server (REST, WebSocket, and a streaming endpoint, in the appendix) so you spend your two days on frontend logic, not on infra. Run it locally. Don't change its behavior. It returns messy data on purpose, and handling that is part of the test.

Note on security: this is a B2B product exercise. The streamed summary contains untrusted content, including some raw HTML. Render it safely. We are watching for whether you sanitize it.

Required stack

- Next.js (App Router) + React 18 + TypeScript (strict mode on; `any` is a smell, justify any use)
- Redux Toolkit for state (slices and selectors; thunks or RTK Query is your call, be ready to defend it)
- Tailwind for styling (plain is fine)
- Jest and React Testing Library for tests

Part 1: Build (the core, about 70% of what I'm looking at)

1. Type the API

The mock `/api/tasks` payload is deliberately messy (see appendix). `type` varies and is sometimes unknown. `status` arrives in inconsistent casing and spelling. Timestamps are

sometimes epoch-ms numbers and sometimes ISO strings. Some counts come back as strings. `assignee` is sometimes null.

- Model the domain properly in TypeScript: a discriminated union on `type`, a normalized status enum, real types for the rest. No `any` dumping ground.
- Write a `normalize.ts` that turns raw payloads into your clean internal model, with narrowing and sane handling of unknown or garbage values. Don't crash, and don't silently drop data. Decide what to do and document it.

2. State (Redux Toolkit)

- A tasks slice holding normalized tasks (createEntityAdapter is a good fit).
- Fetch the initial list (thunk or RTK Query). Handle pagination from the mock.
- Memoized selectors for the derived, filtered, and sorted views. UI components should read selectors, not raw state.

3. Real-time (custom hook)

- A custom hook (something like `useTaskFeed`) that subscribes to the WebSocket stream and dispatches events into state. Events are `task.updated`, `task.assigned`, and `annotation.created` (shapes in the appendix).
- Handle reconnects, and handle the case where an event references a task you haven't loaded yet.

4. UI (function over form)

- A list or table of tasks with sort (by updated time plus one more), filter (by type and status), search, and pagination.
- A detail panel for the selected task.
- Live events visibly merge in. A task's status or assignee updates without a manual refresh.
- Loading, error, empty, and partial states handled deliberately. Show failures, don't hide them.

5. Streamed AI summary (markdown, rendered safely)

- When a task is selected, request its summary from the streaming endpoint (`/api/tasks/:id/summary`, see appendix). The server streams markdown in chunks over a few seconds.
- Render it incrementally as it arrives, not only when the stream finishes. The user should see it build up.
- The content is markdown with code blocks, and it is untrusted. It deliberately includes raw HTML and a script-like payload. Render it as markdown but sanitize it. Do not execute injected HTML or scripts. Be ready to explain how your rendering path is safe.
- Handle the obvious cases: switching tasks mid-stream (cancel the old one), and a stream that errors.

6. Client-side persistence (IndexedDB)

- Cache the most recently loaded task list in IndexedDB (localforage is fine) so that on reload the UI shows the cached data immediately, then revalidates from the server.
- Keep it honest: show that the data is cached or stale until fresh data arrives. Don't block the main thread on large writes.

7. Tests

- At minimum: the normalizer, one selector, and one component interaction with RTL, for example that filtering updates the visible rows.

Bonus (only if the core is solid, and clearly optional)

- Optimistic "assign to me" action with rollback on failure.
- Virtualization for large lists.
- redux-persist for filter and UI state.
- A small derived metric (for example tasks-per-status) shown as a simple chart.
- Cache streamed summaries in IndexedDB so a revisited task shows instantly.

Part 2: Bug hunt (about 15%)

The file `buggy/TaskTicker.tsx` (appendix) is a component a teammate or an AI produced. It mostly works, but it has several real defects. Fix them, and in DECISIONS.md write one to three sentences per bug: the root cause and why your fix is correct. We care more about your explanation than the diff.

There are at least four distinct bugs. Finding more than we planted is a good sign.

Part 3: DECISIONS.md (about 15%, don't skip this)

Keep it short and honest. Cover:

- The key decisions and the tradeoffs behind them (RTK Query vs thunks, your normalization approach, how you typed the messy data, your real-time merge strategy).
- How you render the streamed markdown safely, and exactly where untrusted content is sanitized.
- Your IndexedDB caching approach: what you cache, when you revalidate, how you avoid stale data bugs.
- What you handled for the messy and edge data, and what you deliberately didn't.
- What you'd do differently or do next with more time.
- Anything you used AI for, and how you verified it.

This is the document we'll interview from.

Submission

- A git repo (or zip) that runs with `npm install` then `npm run dev`, plus how to start the mock.

- `npm test` passes.
- A short README: how to run it, and anything we should know.

Appendix A: Mock server

Self-contained Node server: REST, WebSocket, and an SSE streaming endpoint. Drop both files below into a `mock-server/` folder, then from inside it run `npm install` and `npm run mock`. It serves on `http://localhost:4000`, with WS on `ws://localhost:4000/ws`. Requires Node 18 or newer (we tested on Node 22). Do not change its behavior; the messy data and the unsafe stream content are part of the exercise.

`mock-server/package.json`:

```
{
  "name": "mock-server",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "mock": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "ws": "^8.18.0"
  }
}
```

`mock-server/server.js`:

```
// mock-server/server.js
const express = require("express");
const cors = require("cors");
const http = require("http");
const { WebSocketServer } = require("ws");

const app = express();
app.use(cors());
const server = http.createServer(app);

const TYPES = ["image", "audio", "text", "image", "text"]; // 'video' (unknow
n) appears below
const STATUSES = ["in_progress", "InProgress", "done", "QA", "todo", "BLOCKED
"];
const USERS = [
  { id: "u1", name: "Asha" },
  { id: "u2", name: "Ben" },
  { id: "u3", name: "Chen" },
  null, // unassigned happens
];
```

```
// deterministic-ish pseudo data (no real randomness needed)
function makeTask(i) {
  const type = i % 11 === 0 ? "video" /* intentionally unknown type */ : TYPE
S[i % TYPES.length];
  const status = STATUSES[i % STATUSES.length];
  const assignee = USERS[i % USERS.length];
  const useIso = i % 2 === 0;
  const updatedAt = 1719600000000 + i * 37000; // epoch ms
  return {
    id: `t${i}`,
    title: `Task ${i}`,
    type,
    status,                                 // inconsistent casing/spelling on p
urpose
    assignee,                               // sometimes null
    annotationCount: i % 3 === 0 ? String(i) : i,  // sometimes a string on p
urpose
    updatedAt: useIso ? new Date(updatedAt).toISOString() : updatedAt, // mix
ed formats
    meta: i % 4 === 0 ? { priority: "high", note: "rush" } : {},       // fre
e-form
  };
}

const ALL = Array.from({ length: 137 }, (_, i) => makeTask(i + 1));

// GET /api/tasks?page=1&pageSize=20&type=&status=  (server-side paginate onl
y)
app.get("/api/tasks", (req, res) => {
  const page = Math.max(1, parseInt(req.query.page) || 1);
  const pageSize = Math.min(100, parseInt(req.query.pageSize) || 20);
  const start = (page - 1) * pageSize;
  const items = ALL.slice(start, start + pageSize);
  // Occasionally the server is slow, simulate it (tests your loading states)
  const delay = page % 3 === 0 ? 1200 : 200;
  setTimeout(() => {
    res.json({ page, pageSize, total: ALL.length, items });
  }, delay);
});

app.get("/api/tasks/:id", (req, res) => {
  const t = ALL.find((x) => x.id === req.params.id);
  if (!t) return res.status(404).json({ error: "not found" });
  res.json(t);
});

// GET /api/tasks/:id/summary  -> streams a markdown summary in chunks (SSE s
tyle).
```

```
// The content is UNTRUSTED on purpose: it contains raw HTML and a script-lik
e payload.
// Render it as markdown and SANITIZE it. Do not execute the injected HTML/sc
ript.
app.get("/api/tasks/:id/summary", (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.flushHeaders();

  const id = req.params.id;
  const chunks = [
    `## Summary for ${id}\n\n`,
    `This task is **in progress**. Recent activity:\n\n`,
    `- 3 annotations added\n- 1 review pending\n\n`,
    "```ts\nconst score = computeQuality(task); // sample code block\n```\n\n
",
    `Reviewer note: looks good, _ship it_.\n\n`,
    // untrusted content starts here, on purpose:
    `<img src=x onerror="alert('xss-img')">\n\n`,
    `<script>alert('xss-script')</script>\n\n`,
    `Done.\n`,
  ];

  let i = 0;
  const timer = setInterval(() => {
    if (i >= chunks.length) {
      res.write("event: done\ndata: end\n\n");
      clearInterval(timer);
      return res.end();
    }
    // SSE frame; data may contain newlines, so encode it
    res.write(`data: ${JSON.stringify(chunks[i])}\n\n`);
    i += 1;
  }, 400);

  req.on("close", () => clearInterval(timer));
});

// WebSocket: emits a live event roughly every 2s
const wss = new WebSocketServer({ server, path: "/ws" });
wss.on("connection", (ws) => {
  let n = 0;
  const timer = setInterval(() => {
    n += 1;
    const t = ALL[n % ALL.length];
    const kinds = [
      { kind: "task.updated", payload: { id: t.id, status: STATUSES[n % STATU
SES.length], updatedAt: Date.now() } },
```

```
      { kind: "task.assigned", payload: { id: t.id, assignee: USERS[n % USERS
.length] } },
      { kind: "annotation.created", payload: { taskId: t.id, by: "u1", at: Da
te.now() } },
      // references a task that may be beyond the loaded page, handle it grac
efully:
      { kind: "task.updated", payload: { id: `t${120 + (n % 17)}`, status: "d
one", updatedAt: Date.now() } },
    ];
    ws.send(JSON.stringify(kinds[n % kinds.length]));
  }, 2000);
  ws.on("close", () => clearInterval(timer));
});

server.listen(4000, () => console.log("mock on http://localhost:4000 (ws://lo
calhost:4000/ws)"));
```

Appendix B: buggy/TaskTicker.tsx (fix this in Part 2)

A small live counter / recent activity component. It compiles and renders, but it's wrong.

```
// buggy/TaskTicker.tsx
import React, { useEffect, useState } from "react";

type Task = { id: string; title: string; updatedAt: number };

export function TaskTicker({ apiBase }: { apiBase: string }) {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [selectedId, setSelectedId] = useState<string | null>(null);
  const [tick, setTick] = useState(0);

  // (A) keep a running clock for "x seconds ago"
  useEffect(() => {
    const id = setInterval(() => {
      setTick(tick + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []);

  // (B) refetch whenever selection changes
  useEffect(() => {
    fetch(`${apiBase}/api/tasks/${selectedId}`)
      .then((r) => r.json())
      .then((t) => {
        setTasks((prev) => {
          prev.push(t);          // add the freshly-loaded task
          return prev;
        });
```

```
      });
  }, [selectedId]);

  // (C) newest first
  const sorted = tasks.sort((a, b) => b.updatedAt - a.updatedAt);

  return (
    <ul>
      {sorted.map((t, i) => (
        <li key={i} onClick={() => setSelectedId(t.id)}>
          {t.title} (updated {Math.floor((Date.now() - t.updatedAt) / 1000)}s
ago)
        </li>
      ))}
    </ul>
  );
}
```

There’s more than one issue here. We’re looking for root-cause understanding, not just a working render.